Ex. no : 6

DATE : 20/8/24.

ERROR CORRECTION AT DATA LINK LAYER

AIM:

To implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

ALGORITHM:

Sender Program :

1) Start the program
2) Read the input text from input.txt
3) Convert each character of the text into its 8-bit binary representation.
4) Calculate the number of redundant bits required for Hamming Code.
5) Insert redundant bits at position 1, 2, 4, 8...
6) Calculate the parity bits.
7) Generate the Hamming encoded data.
8) Save the encoded data into channel.txt
9) Stop the sender program.

Receiver Program :

1) Start the program
2) Read the encoded data from channel.txt
3) Calculate and check parity bits.
4) Determine the error position using parity check values.

Python sender.py

Output :

```
Input text : HELLO

Binary data : 0100100000100010101001100010011100
              0100111100001010

Hamming encoded data : 000010001000010000 10101001
                       100011001100 01 001111000010 10

Hamming encoded data saved to channel.txt
```

Python receiver.py

Output :

```
Received Hamming data : 000010001000010000101010011
                        000110011000100111100001010

No error detected

Received binary data : 0100100001000101010011000100
                       1000100111100001010

Received message : HELLO
```

Introducing error in channel.txt

Original:

00001 0001 0000 1 00001 0101 0011 000 1 0011 1 000 1001 1111 0000
1010

Changed the first bit from 0 to 1:

10001000 10000 10000 1010100 1 1000 1 0011 1000 1001 1111 0000
1010

Python receiver.py

Output:

Received Hamming data:

10001000 10000 10000 1010100 1 1000 1 0011 1000 1001 1111 0000
1010

Error detected at position: 1

Error corrected successfully

Recovered binary data:

01001 0000 1 000 1 0101 0011 000 1 0011 1000 1001 1111 0000
1010

Received message:

HELLO

5) If an error is detected, display its position.

6) Correct the erroneous bit

7) Remove the redundant bits.

8) Convert the recovered binary data into ASCII text.

9) Display the received message.

10) Stop the receiver program.

Error Correction test:

1) Run the receiver without modifying channel.txt
2) Verify that "No error detected" is displayed and HELLO is received.
3) Change one bit in channel.txt.
4) Run the receiver again.
5) Verify that the error position is displayed
6) Verify that the error is corrected successfully.
7) Verify that the original message HELLO is recovered.

CODE:

Sender.py (Hamming Code for Sender program)

```
def hamming_encode(data):
    m = len(data)
    r=0
    while (2**r) < (m+r+1):
        r+=1
    total_bits = m+r
```

```
code = ['0'] * (total_bits + 1)
j = 0
for i in range (1, total_bits + 1):
    if (i & (i-1)) != 0:
        code [i] = data [j]
        j += 1

for i in range (r):
    position = 2 ** i
    parity = 0
    for j in range (1, total_bits + 1):
        if j & position:
            parity ^ = int(code [j])
    code [position] = str (parity)
return ''.join (code [1:])

with open ("input.txt", "r") as file:
    text = file.read()
binary_data = ''.join (format (ord (char), '08b') for char in text)

print ("Input text :", text)
print ("Binary data :", binary_data)

encoded_data = hamming_encode (binary_data)
print ("Hamming encoded data:").
print (encoded_data)
```

receiver.py (Hammer code for Receiver Program)

```
def hamming_decode (code):
    n = len (code)
    r = 0
    while (2 ** r) < n:
        r += 1
```

```
with open ("channel.txt","r") as file:
    encoded_data = file.read().strip()
print ("Received Hamming data:")
print (encoded_data)

binary_data = hamming_decode (encoded_data)
print ("\nRecovered binary data:")
print (binary_data)

text = ""

for i in range (0, len (binary_data), 8):
    byte = binary_data [i:i+8]
    text += chr (int (byte, 2))
print ("\nReceived message:")
print (text)
```

RESULT:

Therefore, the error detection and correction using HAMMING code has been implemented and executed successfully.